Software Requirement Specs

Metadata

- **Artifact ID**: software_requirement_specs_Archivist_v1.3
- **Version**: 1.3
- **Created By**: Archivist
- **Timestamp**: 2026-03-19T09:03:18.446625+00:00

Reference Artifact Ids

- software_requirement_specs_Archivist_v1.2
- validation_report_Reviewer_v1.1
- system_requirements_Analyst_v1.0
- requirements_model_Analyst_v1.0
- operating_env_list_Deployer_v1.0

Srs Content

Updated SRS for the University Class Scheduler (UCS)

1. Introduction

The University Class Scheduler (UCS) is a strategic, platform-independent academic scheduling system designed to autonomously optimize class-section-to-room assignments while ensuring compliance with regulatory and policy requirements. The system aims to reduce manual adjustments by 70% and provide real-time, explainable, and auditable optimization decisions.

1.1. Background

The University Class Scheduler (UCS) is a deterministic, policy-governed, and FERPA-compliant class scheduling system requiring real-time explainability, versioned constraint weighting, queryable auditability, and auto-populated regulatory documentation. It is designed to reduce manual adjustments by 70% while ensuring traceable, human-reviewable optimization decisions. UCS evolved from legacy manual scheduling processes constrained by inconsistent data sources, post-hoc reporting, and lack of audit-ready rationale generation.

2. General Description

UCS is a role-specific, regulatory-compliant scheduling platform that autonomously assigns class sections to rooms using rule-based optimization. It supports cloud-hosted or on-premises deployment, web-based admin and staff interfaces, and mobile-first UI for students and professors. The system provides real-time scenario simulation, in-context plain-language explanations, FERPA-safe data handling, and auto-generated policy-compliant documentation.

2.1. Product Perspective

UCS is a role-specific, regulatory-compliant scheduling platform that autonomously assigns class sections to rooms using rule-based optimization. It supports cloud-hosted or on-premises deployment, web-based admin and staff interfaces, and mobile-first UI for students and professors. The system provides real-time scenario simulation, in-context plain-language explanations, FERPA-safe data handling, and auto-generated policy-compliant documentation.

2.2. Product Functions

- Autonomous class section assignment with room capacity, departmental proximity, and faculty commute minimization
- Real-time, in-context explanations for constraint relaxation
- Auto-generation of 'Assignment Rationale v2.1' PDF forms
- Persistent, queryable audit trail of scheduling decisions
- Scenario Sandbox for simulating scheduling adjustments
- Personalized timetables for students and professors with offline access and GPS navigation
- Configuration of relative weights for optimization constraints
- Prioritized push notifications for schedule changes and accessibility alerts
- Low-code API console for secure integrations with LMS, CMS, SSO, and SIS systems

2.3. User Classes and Characteristics

- **Department Staff**: Mid-level academic operations personnel responsible for course logistics; goal is efficient departmental scheduling alignment; technically proficient with administrative tools but not developers; interact via web dashboard to review, adjust, and approve auto-generated schedules and sandbox scenarios.
- **University Administrator**: Senior decision-maker overseeing institutional scheduling policy and resource allocation; goal is strategic compliance, equity, and regulatory adherence; moderate technical proficiency; interacts via high-level analytics dashboards, scenario sandbox controls, and audit reports.
- **University Catalogue Creators**: Content specialists who define course attributes (prerequisites, credits, modalities); goal is accurate, maintainable course metadata integration; moderate technical proficiency; interact via structured data entry forms and catalogue validation interfaces.
- **University Webmaster**: IT infrastructure and integration specialist; goal is secure, performant, cross-platform deployment and API connectivity; high technical proficiency (web standards, APIs, CMS); interacts via configuration panels, integration hooks, and monitoring dashboards.
- **Students**: End users consuming finalized class schedules; goal is intuitive access to personal timetables, room locations, and real-time updates; variable technical proficiency (typically comfortable with mobile apps); interact via responsive web and native mobile interfaces with GPS campus mapping and push notifications.
- **Professors**: Academic staff requiring personalized schedule visibility, teaching load balance, and commute-aware assignments; goal is workload transparency and well-being optimization; variable technical proficiency; interact via mobile/web interface to view, request adjustments, receive predictive alerts, and access sandbox impact previews.

2.4. Operating Environment

- **Platform-independent deployment**: Supporting cloud-hosted (AWS/GCP/Azure) or on-premises Linux/Windows servers; containerised (Docker/Kubernetes) preferred; web-based admin and staff interfaces; mobile-first responsive UI for students and professors; offline-capable timetable client for iOS and Android.
- **FERPA-compliant handling**: No ingestion or processing of individual student records—only aggregated, de-identified metrics with full provenance logging; TLS 1.2+ encryption in transit; role-based access control aligned with six defined user classes; digital signature support for approved rationales; audit logs retained per Middle States Commission requirements for systematic, transparent, and periodically reviewable decisions.

2.5. Design and Implementation Constraints

- **Optimization logic**: Must be fully deterministic and rule-based—no ML or probabilistic inference; all constraint weights and trade-off rules must be configurable per college/department without code changes or restarts; explanations must be generated in real time during review; audit trail must support structured querying by policy domain, actor role, college, term, and impact scope; PDF exports must exactly match Faculty Senate Bylaw 7.3 'Assignment Rationale v2.1' template including university letterhead and footer.

2.6. Assumptions and Dependencies

- Users include Department Staff, University Administrators, Professors, Students, and University Webmasters; system used during academic scheduling cycles, with peak demand around draft release and final freeze; chairs make judgment calls based on real-time, in-context explanations—not post-hoc reports.

2.7. Constraints and Assumptions

- **Deterministic Behavior Constraint**: All optimization decisions must be reproducible given identical inputs, weights, and system state; no stochastic or non-deterministic components permitted.
- **Timing Bounds**: All time-bound requirements (e.g., R1 timeout, R2 explanation latency, R4 PDF generation, R6 sandbox visualization, R7 timetable delivery) are qualified with statistical thresholds: ≥95% under peak load for durations ≤10 seconds, ≥99% for durations ≤5 seconds.
- **FERPA Compliance Constraint**: System shall process only aggregated, de-identified enrollment metrics (e.g., historical average ± standard deviation over ≥4 semesters); no PII or individually identifiable data shall enter the optimization engine.
- **API SLA Assumptions**: External data feeds (Facilities DB, GeoJSON maps, room availability WebSockets) are assumed to meet documented SLAs: Facilities DB updated every 15 minutes via webhook; GeoJSON refreshed nightly; WebSocket latency <2s; all payloads verified via SHA-256 checksum.
- **Fallback Assumption**: On external API failure, the system shall automatically retry up to 3 times with exponential backoff, fall back to cached last-known-good data for non-critical integrations (e.g., building map version), and escalate unresolved failures to the University Webmaster within 1 minute.

3. Requirements Specification

The following sections detail the functional and non-functional requirements for the University Class Scheduler (UCS).

3.1. Functional Requirements

- **R1: The system shall implement a platform-independent, deterministic, rule-based optimization engine that autonomously assigns class sections to rooms while balancing room capacity (including accessibility compliance), departmental proximity, and faculty commute minimization, with auditable, configurable weights and trade-off rules per college.**
- **Acceptance Criteria**:
- The system must generate an optimized schedule within 10 minutes in ≥95% of executions under peak load of 500 concurrent users.
- The system must allow for real-time adjustment of weights and rules without service interruption.
- The system must log all optimization decisions and their rationale, including timestamps, actor ID, and applied constraint weights.
- Upon timeout (>10 min), the system shall return the best partial solution generated so far, accompanied by a diagnostic report listing unmet constraints and root causes (e.g., data corruption, missing infrastructure feed), and trigger an alert to University Webmaster.
- **Priority**: High
- **Traceability**:
- SRL: R1
- Use Case: UC-001, UC-008
- User Story: US-001, US-002, US-013, US-014
- **R2: The system shall display real-time, in-context, plain-language explanations adjacent to each auto-generated assignment involving constraint relaxation, addressing: (1) relaxed constraints and justification with data sources and timestamps; (2) top 1–2 realistic alternatives with quantified cost/benefit; and (3) affected stakeholders.**

- **Acceptance Criteria**:
- The system must provide explanations within 5 seconds of an assignment being generated in ≥99% of cases under nominal load.
- The system must present at least two alternative solutions with their respective costs and benefits, sourced from the same optimization run.
- The system must identify and list all affected stakeholders by role and department.
- **Priority**: High
- **Traceability**:
- SRL: R2
- Use Case: UC-001, UC-006
- User Story: US-001, US-002, US-012
- **R4: The system shall auto-generate a completed 'Assignment Rationale v2.1' PDF form upon Department Staff approval or override of an assignment, pre-filled with decision date, UCS version, course/section ID, relaxed constraints, cited rationale with data source/timestamp, and digital signature field, matching university letterhead and footer formatting exactly. The PDF shall support multilingual text (English, Spanish, French), conform to WCAG 2.1 AA for screen reader compatibility, and render dynamic content (e.g., variable-length course titles) without truncation or overflow.**
- **Acceptance Criteria**:
- The system must generate the PDF within 10 seconds in ≥95% of executions under peak load.
- The PDF must match the exact format specified in the Faculty Senate Bylaw 7.3, including all visual elements and structural metadata.
- The PDF must include a digital signature field and support embedded language tags and ARIA labels for assistive technologies.
- **Priority**: High
- **Traceability**:
- SRL: R4
- Use Case: UC-001
- User Story: US-001, US-004
- **R5: The system shall maintain a persistent, queryable audit trail of all scheduling decisions involving constraint relaxations, supporting filtering by policy domain, actor role, college, term, and impact scope, with export capability aligned with Middle States Commission requirements.**
- **Acceptance Criteria**:
- The system must log all scheduling decisions and their associated metadata, including immutable cryptographic hash of each record.
- The system must support filtering and exporting of audit logs in CSV and PDF formats, with timestamped digital signatures on exported reports.
- The exported logs must meet the Middle States Commission requirements for systematic, transparent, and periodically reviewable decisions.
- **Priority**: High
- **Traceability**:
- SRL: R5
- Use Case: UC-001, UC-007

- User Story: US-003, US-004
- **R6: The system shall provide a Scenario Sandbox enabling Department Staff and Professors to simulate scheduling adjustments and instantly visualize predicted impacts on faculty commute time/distance, departmental proximity violations, and room capacity/accessibility compliance using live campus infrastructure data sourced from Facilities DB (REST API, updated every 15 minutes via webhook), GPS-enabled building maps (versioned GeoJSON, refreshed nightly), and real-time room availability feeds (WebSockets, <2s latency). Data integrity is guaranteed via SHA-256 checksum verification on all inbound payloads.**
- **Acceptance Criteria**:
- The system must allow for real-time simulation of scheduling adjustments with no user-perceived lag.
- The system must visualize the predicted impacts within 5 seconds in ≥99% of executions.
- The system must use live campus infrastructure data as defined above, with documented SLAs for freshness and integrity.
- **Priority**: High
- **Traceability**:
- SRL: R6
- Use Case: UC-001, UC-006, UC-008
- User Story: US-002, US-012, US-014
- **R7: The system shall deliver personalized timetables to Students and Professors via mobile and web interfaces featuring real-time sync, offline caching of current term data, GPS-integrated accessibility-aware navigation (elevator/ramp paths), and preloaded route maps, with seamless sync upon reconnection.**
- **Acceptance Criteria**:
- The system must deliver timetables within 10 seconds in ≥95% of requests under peak load.
- The system must support offline caching and real-time sync, preserving local state during network outages and resolving conflicts via last-write-wins with timestamp reconciliation.
- The system must provide GPS-integrated accessibility-aware navigation, including step-free path routing validated against campus ADA compliance database.
- **Priority**: High
- **Traceability**:
- SRL: R7
- Use Case: UC-005, UC-006
- User Story: US-009, US-011, US-015
- **R8: The system shall allow University Administrators to configure, version, and enforce relative weights for optimization constraints (capacity, proximity, commute) at the college or department level without code changes or system restarts.**
- **Acceptance Criteria**:
- The system must allow for real-time configuration of weights with immediate effect on subsequent optimization runs.
- The system must version and enforce the configured weights, retaining historical versions for audit and rollback.
- The system must not require code changes or system restarts for weight updates.
- **Priority**: High
- **Traceability**:

- SRL: R8
- Use Case: UC-002, UC-007
- User Story: US-003, US-004
- **R9: The system shall send prioritized push notifications to Students for schedule changes, room reassignments, or accessibility alerts—categorized by urgency and relevance—with per-type opt-in/opt-out and suppression during declared maintenance windows.**
- **Acceptance Criteria**:
- The system must send notifications within 5 seconds of a change in ≥99% of cases.
- The system must categorize notifications by urgency (Critical, High, Medium, Low) and relevance (Personal, Departmental, Campus-wide), applying configurable delivery throttling rules.
- The system must support per-type opt-in/opt-out and suppression during maintenance windows declared via the Admin Dashboard.
- **Priority**: High
- **Traceability**:
- SRL: R9
- Use Case: UC-005
- User Story: US-009, US-010
- **R10: The system shall provide a low-code API console for University Webmasters to configure, test, monitor, and secure integrations with LMS, CMS, SSO, and SIS systems using OAuth2/SAML, payload mapping, latency/uptime monitoring, and encryption status visibility—without database access or custom scripting. On external API failure, the system shall automatically retry up to 3 times with exponential backoff, fall back to cached last-known-good data for non-critical integrations, and escalate unresolved failures to the University Webmaster via email and dashboard alert within 1 minute.**
- **Acceptance Criteria**:
- The system must provide a low-code API console with drag-and-drop mapping, real-time test console, and integrated monitoring dashboard.
- The system must support OAuth2/SAML, payload mapping, latency/uptime monitoring, and encryption status visibility.
- The system must not require database access or custom scripting and must implement robust error recovery as specified.
- **Priority**: High
- **Traceability**:
- SRL: R10
- Use Case: UC-004
- User Story: US-007, US-008

3.2. Non-functional Requirements

- **R3: The system shall process only aggregated, de-identified enrollment metrics (e.g., historical average ± standard deviation over ≥4 semesters) as inputs to the optimization engine and shall log the exact metric source, aggregation method, and timestamp for each input used.**
- **Acceptance Criteria**:
- The system must use only aggregated, de-identified enrollment metrics.

- The system must log the exact metric source, aggregation method, and timestamp for each input used, with immutable linkage to the optimization run ID.
- **Priority**: High
- **Traceability**:
- SRL: R3
- Use Case: UC-001, UC-008
- User Story: US-001, US-014

4. Use Case Modeling

The following sections detail the use case diagram, detailed use cases, and scenarios for the University Class Scheduler (UCS).

4.1. Use Case Diagram

```
@startuml
actor "Department Staff" as DeptStaff
actor "Professor" as Prof
actor "Student" as Student
actor "University Administrator" as Admin
actor "University Webmaster" as Webmaster

package "Class Scheduling Optimization System" {
usecase "Optimize Class Section Assignments" as UC_Optimize
usecase "Explain Constraint Relaxation" as UC_Explain
usecase "Generate Assignment Rationale PDF" as UC_PDF
usecase "Maintain Audit Trail" as UC_Audit
usecase "Run Scenario Sandbox Simulation" as UC_Sandbox
usecase "Deliver Personalized Timetables" as UC_Timetables
usecase "Configure Optimization Weights" as UC_ConfigureWeights
usecase "Send Prioritized Push Notifications" as UC_Notifications
usecase "Manage Low-Code API Integrations" as UC_API
}

DeptStaff --> UC_Optimize
DeptStaff --> UC_Explain
DeptStaff --> UC_PDF
DeptStaff --> UC_Sandbox
Prof --> UC_Sandbox
Prof --> UC_Timetables
Student --> UC_Timetables
Student --> UC_Notifications
Admin --> UC_ConfigureWeights
Admin --> UC_Audit
Webmaster --> UC_API

UC_Optimize ..> UC_Explain : <<include>>
UC_Optimize ..> UC_Audit : <<include>>
UC_Optimize ..> UC_ConfigureWeights : <<include>>
UC_Sandbox ..> UC_Timetables : <<include>>
UC_PDF ..> UC_Audit : <<include>>

note right of UC_Optimize : R1, R3 (de-identified inputs)
note right of UC_Explain : R2
```

```
note right of UC_PDF : R4
note right of UC_Audit : R5
note right of UC_Sandbox : R6
note right of UC_Timetables : R7
note right of UC_ConfigureWeights : R8
note right of UC_Notifications : R9
note right of UC_API : R10
@enduml
```

4.2. Detailed Use Cases

- **UC-001: Approve Auto-Generated Departmental Schedule**
- **Actor**: Department Staff
- **Description**: Department Staff reviews an auto-generated schedule in the dashboard, verifies room capacity, departmental proximity, and faculty commute metrics, and approves or requests adjustments.
- **UC-002: Configure Regulatory Compliance Rules**
- **Actor**: University Administrator
- **Description**: University Administrator accesses the Policy Configuration Interface to define and activate FERPA, GDPR, and accreditation-specific constraints that govern scheduling logic and reporting outputs.
- **UC-003: Edit and Validate Course Catalogue Metadata**
- **Actor**: University Catalogue Creators
- **Description**: University Catalogue Creators use the Course Metadata Editor to input or update course attributes, triggering real-time validation against prerequisite chains, modality rules, and credit-hour policies.
- **UC-004: Integrate UCS with External Systems via API Console**
- **Actor**: University Webmaster
- **Description**: University Webmaster uses the API Configuration Console to authenticate, map endpoints, test payloads, and enable bi-directional sync with LMS, CMS, and SSO systems using only documented RESTful APIs.
- **UC-005: View and Navigate Personal Timetable Offline**
- **Actor**: Students
- **Description**: Students open the mobile interface to view their cached timetable; upon network restoration, pending changes sync automatically while preserving offline usability and GPS route preloading.
- **UC-006: Simulate Impact of Faculty Adjustment Request**
- **Actor**: Professors
- **Description**: Professors submit a class assignment change request and immediately view sandbox-generated forecasts for commute time delta, teaching load redistribution, and constraint violations across departmental and regulatory dimensions.
- **UC-007: Generate Equity and Compliance Audit Report**
- **Actor**: University Administrator
- **Description**: University Administrator selects reporting parameters (e.g., term, department, equity metric) and generates an exportable, timestamped report showing statistical analysis of schedule fairness, accessibility coverage, and regulatory alignment.
- **UC-008: Run Bottleneck Prediction for Enrollment Surge Scenario**
- **Actor**: University Administrator
- **Description**: University Administrator initiates a 'What-If' simulation in the Scenario Sandbox modeling a 15% enrollment increase, triggering the Bottleneck Prediction Engine to identify overcapacity rooms, strained departments, and faculty commute hotspots with resolution recommendations.

4.3. Scenarios

- **Scenario 1: Department Staff reviewing and approving auto-generated schedules**
- **Scenario 2: University Administrator configuring regulatory compliance rules**
- **Scenario 3: University Catalogue Creators editing and validating course metadata**
- **Scenario 4: University Webmaster integrating UCS with external systems via API console**
- **Scenario 5: Students viewing and navigating personal timetables offline**
- **Scenario 6: Professors simulating the impact of adjustment requests**
- **Scenario 7: University Administrator generating equity and compliance audit reports**
- **Scenario 8: University Administrator running bottleneck prediction for enrollment surge scenarios**

5. Class Diagram

Embedded UML Class Diagram artifact referenced as `ucs_class_diagram_v1.0` *— available at https://artifacts.university.edu/ucs/ucs_class_diagram_v1.0.puml. Key entities include: CourseSection, Room, Faculty, Department, ConstraintWeight, AuditLogEntry, ScenarioSandbox, and AssignmentRationale. Relationships reflect cardinality, inheritance (e.g., Room → AccessibilityRoom), and constraints (e.g., CourseSection.room_id → Room.id, non-nullable). All attributes are typed and visibility-scoped.*

6. Traceability Matrix

Requirement ID	Source Document	Stakeholder(s)	Use Case ID(s)	Design Element	Test Case ID
R1	SRL-001, US-001, US-014	Department Staff, University Administrator	UC-001, UC-008	Scheduler Module v2.1	TC-R1-001
R2	SRL-002, US-001, US-012	Department Staff, Professors	UC-001, UC-006	Explanation Engine v1.3	TC-R2-001
R3	SRL-003, US-001, US-014	University Administrator	UC-001, UC-008	Input Validation Layer	TC-R3-001
R4	SRL-004, US-001, US-004	Department Staff	UC-001	PDF Generator v2.1	TC-R4-001
R5	SRL-005, US-003, US-004	University Administrator	UC-001, UC-007	Audit Log Service v1.0	TC-R5-001
R6	SRL-006, US-002, US-012	Department Staff, Professors	UC-001, UC-006, UC-008	Scenario Sandbox Core	TC-R6-001
R7	SRL-007, US-009, US-015	Students, Professors	UC-005, UC-006	Timetable Sync Engine v1.2	TC-R7-001
R8	SRL-008, US-003, US-004	University Administrator	UC-002, UC-007	Weight Configuration Service v1.1	TC-R8-001
R9	SRL-009, US-009, US-010	Students	UC-005	Notification Orchestrator v1.0	TC-R9-001
R10	SRL-010, US-007, US-008	University Webmaster	UC-004	API Console v1.4	TC-R10-001

7. Appendix A: Data Specifications

(a) Input Payload Schema (Facilities DB Feed)

- Format: JSON Schema v7

- Root object: `FacilitiesData`
- Fields: `building_id` (string, required), `room_id` (string, required), `capacity` (integer, ≥0), `accessibility_features` (array of strings, enum: ["elevator", "ramp", "automatic_door", "braille_signage"]), `last_updated` (ISO 8601 datetime, required), `checksum` (string, SHA-256 hex, required)
- Quality Rule: Null tolerance = 0%; precision = integer for capacity, millisecond for timestamp.

(b) Output PDF Metadata Spec

- Format: PDF/A-2u compliant
- Required Metadata: `Title`, `Author` (UCS v2.1), `Subject` ("Assignment Rationale v2.1"), `Creator` (UCS PDF Generator), `CreationDate`, `ModDate`, `XMP` namespace with `university:letterheadURL`, `university:footerText`, `ucssystem:version`, `ucssystem:runID`
- Accessibility: Embedded language tags (en, es, fr), ARIA labels for all interactive fields, tagged PDF structure tree.

(c) Audit Log Field Definitions & Retention Rules

- Fields: `log_id` (UUID), `timestamp` (ISO 8601), `actor_role` (enum), `college_code` (string), `term_code` (string), `policy_domain` (enum), `impact_scope` (enum), `constraint_relaxed` (array), `rationale_text` (string), `data_source` (string), `hash` (SHA-256), `export_signature` (base64 DER)
- Retention: Immutable storage for 7 years; exportable in CSV/PDF with timestamped digital signature per Middle States Commission §4.2.1.

8. Current Status

The system is currently in the design phase, with detailed requirements and use cases defined. The next steps include the development of the core scheduling engine, user interfaces, and integration with external systems.

9. Conclusions

The University Class Scheduler (UCS) is a comprehensive, autonomous, and compliant academic scheduling system designed to streamline class-section-to-room assignments. It provides real-time, explainable, and auditable optimization decisions, reducing manual adjustments and ensuring regulatory compliance.

Summary

The Software Requirements Specification (SRS) for the University Class Scheduler (UCS) details the system's background, general description, user classes, operating environment, functional and non-functional requirements, use case modeling, traceability matrix, data specifications, and current status. The SRS addresses all validation issues: (1) added dedicated Constraints and Assumptions section (2.7); (2) added formal Traceability Matrix (Section 6); (3) added Appendix A with detailed data schemas, quality criteria, and retention rules; (4) preserved all original requirement content, traceability links, and acceptance criteria; (5) maintained strict IEEE 830/ISO 29148 structural compliance.

Reference Artifact IDs

- `software_requirement_specs_Archivist_v1.2`
- `validation_report_Reviewer_v1.1`
- `system_requirements_Analyst_v1.0`
- `requirements_model_Analyst_v1.0`
- `operating_env_list_Deployer_v1.0`

Summary

Revised SRS addresses all validation issues: (1) Added dedicated 'Constraints and Assumptions' section (2.7) with deterministic behavior, timing bounds, FERPA compliance, API SLAs, and fallback assumptions; (2) Added formal Traceability Matrix (Section 6) in table format linking R1–R10 to source documents, stakeholders, use cases, design elements, and test cases; (3) Added

Appendix A: Data Specifications with JSON schema for Facilities DB feed, PDF metadata spec, and audit log field definitions/retention rules; (4) Preserved all original requirements, acceptance criteria, and traceability references; (5) Maintained full IEEE 830/ISO 29148 structural compliance.

Sections

None

Template Artifact Id

srs_template_User_v1.0